

Amazon DynamoDB

A fast and flexible NoSQL database service for any scale

DynamoDB is a fully managed, key-value, and document database that delivers single-digit-millisecond performance at any scale.

What is NoSQL?

NoSQL is a type of database designed to store and manage data in flexible, non-tabular formats, making it ideal for handling large volumes of unstructured data.

```
{  
  "id": 1  
  "name": "Raju",  
  "email": "raju@example.com"  
}
```

DynamoDB stores data as items in tables, with each item represented as a JSON-like document consisting of key-value pairs.

DynamoDB also offers a free tier that provides 25 GB of storage.

The free tier also includes 25 provisioned Write /Read Capacity Units (WCU, RCU) which is enough to handle 200 M requests per month.

- **Serverless:** No need for server provisioning, software installation, maintenance, or patching.
- **Automatic Scaling:** Instantly scales up or down based on demand, with no manual adjustments needed.
- **Zero Downtime:** Provides continuous availability without maintenance windows.
- **On-Demand Pricing:** Pay only for the read/write requests used, ideal for fluctuating workloads.
- **Idle Cost Savings:** Scales down to zero during inactivity, so there's no cost when tables have no traffic.

Part 1 — DynamoDB Table Design

Create a DynamoDB table named: student_performance

Attributes:

student_id
weekly_self_study_hours
attendance_percentage
class_participation
total_score
grade
performance_category

Primary Key Partition Key: student_id

Done with the help of aws gui creating table and setting student id as partition key - it treats it as primary key

On aws **dynamodb console** to the table then in right corner **explore table items** and **create item** and create and edit attributes.

Part 2 — Performance Category Calculation

Your Lambda function must calculate a new attribute called: **performance_category**

Rules Total Score Category

>= 90 Excellent

75 – 89 Good

60 – 74 Average

< 60 Poor

Sol- inside lambda folder

Created lambda [function.py](#)

```
def calculate_performance_category(total_score):  
    """Calculate performance category based on total score"""  
  
    if total_score >= 90:  
        return "Excellent"  
    elif total_score >= 75:  
        return "Good"  
    elif total_score >= 60:  
        return "Average"  
    else:  
        return "Poor"
```

Part 3 — S3 Upload

Create an S3 bucket.

Example:

student-performance-datasets

Upload JSON files containing student records.

Configure S3 event notification to trigger a Lambda function whenever a file is uploaded.

Flow:

JSON File Upload

|

v

Amazon S3 Bucket

|

v

S3 Event Notification

|

v

AWS Lambda Trigger

Whenever a JSON file is uploaded:

- S3 automatically triggers Lambda.

This is called:

Event-Driven Architecture

Sol - In this created s3 bucket and uploaded json data to s3 bucket .

Part 4 — Lambda Processing

Write a Python Lambda function that:

1. Reads the uploaded file from S3
2. Parses the JSON data
3. Calculates performance_category
4. Inserts records into DynamoDB The function should handle multiple records in a single file

What Happens End-to-End

sample_students.json

|

▼

Amazon S3

|

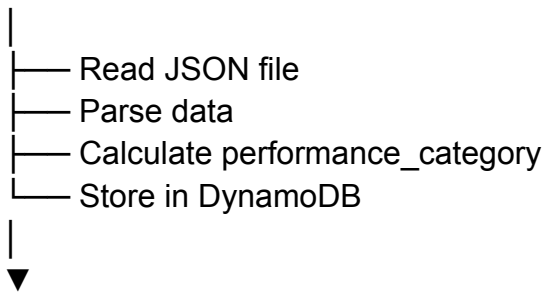
▼

S3 Event Notification

|

▼

AWS Lambda



student_performance Table

Step 1 — Lambda Triggered Automatically

When you uploaded:

sample_students.json

to S3, S3 generated an:

ObjectCreated Event

and automatically invoked Lambda.

AWS passed an event object to Lambda.

Example:

`event["Records"][0]`

contains:

```
{
  "s3": {
    "bucket": {
      "name": "student-performance-datasets"
    },
    "object": {
      "key": "sample_students.json"
    }
  }
}
```

This tells Lambda:

Which bucket?

Which file?

Step 2 — Read File From S3

You created an S3 client:

```
s3 = boto3.client("s3")
```

Then:

```
response = s3.get_object(  
    Bucket=bucket_name,  
    Key=file_key  
)
```

This downloads the uploaded file.

Think of it as:

Open file from S3

↓

Read contents

↓

Store in memory

Step 3 — Parse JSON

The file content arrives as text.

Example:

```
[  
  {  
    "student_id": 1,  
    "total_score": 97.9
```

```
}  
]
```

You converted it into Python objects:

```
students = json.loads(  
    file_content,  
    parse_float=Decimal  
)
```

After parsing:

```
students[0]
```

becomes:

```
{  
    "student_id": 1,  
    "total_score": Decimal("97.9")  
}
```

Now Python can work with the data.

Step 4 — Calculate performance_category

You created:

```
def calculate_performance_category(score):
```

Business Rules:

Score	Category
-------	----------

>= 90	Excellent
-------	-----------

75–89	Good
-------	------

60–74 Average

< 60 Poor

Example:

97.9

returns:

Excellent

Then you added it to the record:

```
student["performance_category"] = category
```

Result:

```
{  
  "student_id": "1",  
  "total_score": Decimal("97.9"),  
  "performance_category": "Excellent"  
}
```

This process is called:

Data Enrichment

because you're adding new information before storing the data.

Step 5 — Convert student_id

Your DynamoDB table was created with:

Partition Key:

student_id (String)

But the dataset contained:

student_id = 1

which is a Number.

So you converted:

```
student["student_id"] = str(
    student["student_id"]
)
```

Result:

"1"

Now the key type matches DynamoDB.

Step 6 — Handle DynamoDB Number Format

DynamoDB doesn't like Python floats.

Example:

```
97.9
18.5
95.6
```

must be stored as:

```
Decimal("97.9")
```

You solved that by:

```
json.loads(
    file_content,
    parse_float=Decimal
)
```

This automatically converts every decimal number.

Step 7 — Insert Into DynamoDB

You created:

```
dynamodb = boto3.resource("dynamodb")
```

Connected to:

```
table = dynamodb.Table(  
    "student_performance"  
)
```

Then:

```
table.put_item(  
    Item=student  
)
```

The Lambda function was triggered automatically whenever a JSON dataset was uploaded to Amazon S3. It extracted the bucket name and file name from the S3 event, downloaded the uploaded file, parsed the JSON records, calculated a derived attribute called `performance_category` based on each student's `total_score`, and inserted the enriched records into the DynamoDB `student_performance` table. The function was designed to process multiple student records in a single file and included error handling for robust execution.

Here facing issue that data was unable to save to dynamodb so have given that access through iam console .

Part 5 — CRUD Operations

Write Python scripts to perform the following operations using DynamoDB. Create Insert a new student record.

Read Retrieve student performance data using: `student_id`

Update Update any of the following fields:

`weekly_self_study_hours`

`attendance_percentage`

`class_participation`

`total_score` `grade`

Delete Delete a student record

```
class StudentCRUD:

    def __init__(self):

        self.dynamodb = boto3.resource(
            "dynamodb",
            region_name="ap-south-1"
        )

        self.table = self.dynamodb.Table(
            "student_performance"
        )

        # -----
        # CREATE
        # -----

    def create_student(self, student):

        response = self.table.put_item(
            Item=student
        )

        print("Student inserted successfully")

        return response
```

```
# -----
# READ
# -----

    def read_student(self, student_id):

        response = self.table.get_item(
            Key={
                "student_id": str(student_id)
            }
        )
```

```
item = response.get("Item")

if item:
    print(item)
else:
    print("Student not found")

return item
```

```
# -----
# UPDATE
# -----

def update_student(
    self,
    student_id,
    weekly_self_study_hours,
    attendance_percentage,
    class_participation,
    total_score,
    grade
):

    response = self.table.update_item(

    Key={
        "student_id": str(student_id)
    },

    UpdateExpression="""
    SET
    weekly_self_study_hours = :h,
    attendance_percentage = :a,
    class_participation = :c,
    total_score = :s,
    grade = :g
    """,

    ExpressionAttributeValues={
```

```

        ":h": Decimal(
            str(weekly_self_study_hours)
        ),

        ":a": Decimal(
            str(attendance_percentage)
        ),

        ":c": Decimal(
            str(class_participation)
        ),

        ":s": Decimal(
            str(total_score)
        ),

        ":g": grade
    },

    ReturnValues="UPDATED_NEW"
)

print("Student updated")

return response

```

```

# -----
# DELETE
# -----

def delete_student(
    self,
    student_id
):

    response = self.table.delete_item(
        Key={

```

```

"student_id": str(student_id)
}
)

print("Student deleted")

return response

```

Part 6 — Query Operations

Implement the following queries.

Query 1 Retrieve students with:

attendance_percentage > 90

Query 2 Retrieve students who studied more than:

weekly_self_study_hours > 10

Query 3 Retrieve students in the Excellent performance category

```

import boto3
from boto3.dynamodb.conditions import Attr

```

```

class StudentQueries:

```

```

    def __init__(self):

        self.dynamodb = boto3.resource(
            "dynamodb",
            region_name="ap-south-1"
        )

```

```

        self.table = self.dynamodb.Table(
            "student_performance"
        )

```

```

# -----
# Query 1
# attendance_percentage > 90
# -----

```

```

    def attendance_above_90(self):

```

```

response = self.table.scan(
    FilterExpression=
    Attr("attendance_percentage").gt(90)
)

students = response["Items"]

print(
    f"\nStudents with attendance > 90: "
    f"{len(students)}\n"
)

for student in students:
    print(student)

return students

# -----
# Query 2
# weekly_self_study_hours > 10
# -----

def study_hours_above_10(self):

    response = self.table.scan(
        FilterExpression=
        Attr(
            "weekly_self_study_hours"
        ).gt(10)
    )

    students = response["Items"]

    print(
        f"\nStudents with study hours > 10: "
        f"{len(students)}\n"
    )

    for student in students:

```

```

        print(student)

    return students

# -----
# Query 3
# Excellent Students
# -----

def excellent_students(self):

    response = self.table.scan(
        FilterExpression=
        Attr(
            "performance_category"
        ).eq("Excellent")
    )

    students = response["Items"]

    print(
        f"\nExcellent Students: "
        f"{len(students)}\n"
    )

    for student in students:
        print(student)

    return students

```

Test query

```

from student_queries import StudentQueries

query = StudentQueries()

while True:

    print("\n")
    print("1. Attendance > 90")
    print("2. Study Hours > 10")

```

```
print("3. Excellent Students")
print("4. Exit")

choice = input("\nEnter Choice: ")

if choice == "1":
    query.attendance_above_90()

elif choice == "2":
    query.study_hours_above_10()

elif choice == "3":
    query.excellent_students()

elif choice == "4":
    break

else:
    print("Invalid Choice")
```

Part 7 — Secondary Index

Create a Global Secondary Index (GSI).

Example:

Partition Key: grade

Sort Key: total_score

Use this index to retrieve:

- Top students in grade A
- Students with highest scores

What is a GSI?

A GSI creates another access pattern.

Current:

student_id

New GSI:

grade

```
+  
total_score
```

Now DynamoDB can directly answer:

Give me Grade A students

sorted by score

Final code for part 6 and part 7 ----sol ----

```
import boto3  
from boto3.dynamodb.conditions import Attr, Key
```

```
class StudentQueries:
```

```
    def __init__(self):
```

```
        self.dynamodb = boto3.resource(  
            "dynamodb",  
            region_name="ap-south-1"  
        )
```

```
        self.table = self.dynamodb.Table(  
            "student_performance"  
        )
```

```
    # -----
```

```
    # Query 1
```

```
    # attendance_percentage > 90
```

```
    # -----
```

```
    def attendance_above_90(self):
```

```
        response = self.table.scan(  
            FilterExpression=Attr(  
                "attendance_percentage"  
            ).gt(90)  
        )
```

```
        students = response["Items"]
```

```

    print(
        f"\nStudents with attendance > 90: "
        f"{len(students)}\n"
    )

    for student in students:
        print(student)

    return students

# -----
# Query 2
# weekly_self_study_hours > 10
# -----

def study_hours_above_10(self):

    response = self.table.scan(
        FilterExpression=Attr(
            "weekly_self_study_hours"
        ).gt(10)
    )

    students = response["Items"]

    print(
        f"\nStudents with study hours > 10: "
        f"{len(students)}\n"
    )

    for student in students:
        print(student)

    return students

# -----
# Query 3
# Excellent Students
# -----

```

```

def excellent_students(self):

    response = self.table.scan(
        FilterExpression=Attr(
            "performance_category"
        ).eq("Excellent")
    )

    students = response["Items"]

    print(
        f"\nExcellent Students: "
        f"{len(students)}\n"
    )

    for student in students:
        print(student)

    return students

```

```

# -----
# Part 7
# Top Students in Grade A
# Uses GSI:
# grade-score-index
# -----

```

```

def top_students_grade_a(self):

    response = self.table.query(

        IndexName="grade-score-index",

        KeyConditionExpression=
            Key("grade").eq("A"),

        ScanIndexForward=False,

        Limit=10

```

```

)

students = response["Items"]

print("\nTop Grade A Students\n")

for student in students:
    print(student)

return students

```

```

# -----
# Part 7
# Highest Scoring Students
# -----

```

```

def highest_scores(self):

    response = self.table.scan()

    students = response["Items"]

    students.sort(
        key=lambda x: float(x["total_score"]),
        reverse=True
    )

    top_students = students[:10]

    print("\nHighest Scoring Students\n")

    for student in top_students:
        print(student)

    return top_students

```

```

# -----
# Main Menu
# -----

```

```
if __name__ == "__main__":

    query = StudentQueries()

    while True:

        print("\n")
        print("=" * 50)
        print("STUDENT PERFORMANCE QUERY MENU")
        print("=" * 50)

        print("1. Attendance > 90")
        print("2. Study Hours > 10")
        print("3. Excellent Students")
        print("4. Top Grade A Students")
        print("5. Highest Scoring Students")
        print("6. Exit")

        choice = input("\nEnter Choice: ")

        if choice == "1":

            query.attendance_above_90()

        elif choice == "2":

            query.study_hours_above_10()

        elif choice == "3":

            query.excellent_students()

        elif choice == "4":

            query.top_students_grade_a()

        elif choice == "5":

            query.highest_scores()
```

```
elif choice == "6":  
  
    print("Exiting...")  
    break  
  
else:  
  
    print("Invalid Choice")
```

Part 8 — Data Import

Upload a dataset containing at least 200 student records to S3.

Verify that the Lambda function processes the data and stores it in DynamoDB.

Sol- so here what was happening that through lambda function there is trigger in lambda so when any thing is uploaded to s3 lambda function automatically invoked and Take data from that file and parse that json data and save it to the dynamodb that was Automation pipeline .

Issue - data was no inserted and request time out due to time limit or time to process was 3 sec in lambda configuration so increased tahat time limit to 30 sec and issue resolved now .

Part 9 — Data Export Export student performance records from DynamoDB into a file.

Supported formats: JSON CSV Upload the exported file to S3.

```
import  
import csv  
import boto3  
  
class ExportData:
```

```

def __init__(self):

    self.dynamodb = boto3.resource(
        "dynamodb",
        region_name="ap-south-1"
    )

    self.table = self.dynamodb.Table(
        "student_performance"
    )

    self.s3 = boto3.client(
        "s3",
        region_name="ap-south-1"
    )

    self.bucket_name = (
        "student-performance-datasets-ayush"
    )

# -----
# Get All Students
# -----

def get_all_students(self):

    response = self.table.scan()

    return response["Items"]

# -----
# Export JSON
# -----

def export_json(self):

    students = self.get_all_students()

    file_name = (

```

```

        "students_export.json"
    )

    with open(
        file_name,
        "w"
    ) as file:

        json.dump(
            students,
            file,
            indent=4,
            default=str
        )

    print(
        f"{file_name} created"
    )

    return file_name

# -----
# Export CSV
# -----

def export_csv(self):

    students = self.get_all_students()

    if not students:

        print(
            "No records found"
        )

        return None

    file_name = (
        "students_export.csv"
    )

```

```

        with open(
            file_name,
            "w",
            newline=""
        ) as file:

            writer = csv.DictWriter(
                file,
                fieldnames=
                    students[0].keys()
            )

            writer.writeheader()

            writer.writerows(
                students
            )

        print(
            f"{file_name} created"
        )

        return file_name

# -----
# Upload to S3
# -----

def upload_to_s3(
    self,
    file_name
):

    self.s3.upload_file(

        file_name,

        self.bucket_name,

```

```
        f"exports/{file_name}"
    )

    print(
        f"{file_name} uploaded to S3"
    )

if __name__ == "__main__":

    exporter = ExportData()

    while True:

        print("\n")
        print("=" * 50)
        print("EXPORT MENU")
        print("=" * 50)

        print("1. Export JSON")
        print("2. Export CSV")
        print("3. Exit")

        choice = input(
            "\nEnter Choice: "
        )

        if choice == "1":

            file_name = (
                exporter.export_json()
            )

            exporter.upload_to_s3(
                file_name
            )

        elif choice == "2":

            file_name = (
```

```

        exporter.export_csv()
    )

    exporter.upload_to_s3(
        file_name
    )

elif choice == "3":

    break

else:

    print(
        "Invalid Choice"
    )

```

I implemented a data export module that retrieves student performance records from DynamoDB, converts them into JSON and CSV formats, and uploads the generated files to an exports folder in Amazon S3. The export process is separated from the ingestion pipeline by using dedicated S3 prefixes (`input/` and `exports/`) to prevent Lambda from reprocessing exported files.

Inside Lambda you created:

```

def calculate_risk(
    attendance,
    score
):

    if (
        attendance < 60
        or score < 50
    ):

        return True

    return False

```

Then during data processing:

```
risk = calculate_risk(
    float(
        student["attendance_percentage"]
    ),
    float(
        student["total_score"]
    )
)

student["at_risk"] = risk
```

I implemented a risk detection mechanism in the Lambda function. During data ingestion, the function evaluates attendance percentage and total score. Students with attendance below 60% or total score below 50 are flagged with an `at_risk` attribute and stored in DynamoDB for further monitoring and intervention.

```
def leaderboard(self):
```

Steps:

Read all records

```
response = self.table.scan()
```

Sort by total_score

```
students.sort(
    key=lambda x:
        float(x["total_score"]),
    reverse=True
)
```

Take Top 10

```
leaderboard = students[:10]
```

I implemented a leaderboard feature that retrieves student records from DynamoDB, sorts them by total score in descending order, and displays the top 10 performers. This provides a quick view of the highest-achieving students and supports academic performance analysis.

Advanced Task 3

Implemented here:

if len(students) > 500:

with table.batch_writer() as batch:

✓ Completed

Advanced Task 4

Invalid JSON

except json.JSONDecodeError:

✓ Completed

Missing Fields

missing_fields = [...]

✓ Completed

```
import
import boto3
from decimal import Decimal

# AWS Clients
s3 = boto3.client("s3")

dynamodb = boto3.resource("dynamodb")

table = dynamodb.Table(
    "student_performance"
```

```

)

# -----
# Performance Category
# -----

def calculate_performance_category(score):

    if score >= 90:
        return "Excellent"

    elif score >= 75:
        return "Good"

    elif score >= 60:
        return "Average"

    else:
        return "Poor"

# -----
# Student Risk Detection
# -----

def calculate_risk(
    attendance,
    score
):

    if (
        attendance < 60
        or score < 50
    ):
        return True

    return False

```

```
# -----
# Lambda Handler
# -----

def lambda_handler(event, context):

    try:

        # -----
        # Get Bucket & File Name
        # -----

        bucket_name = (
            event["Records"][0]
            ["s3"]["bucket"]["name"]
        )

        file_key = (
            event["Records"][0]
            ["s3"]["object"]["key"]
        )

        print(
            f"Bucket Name: {bucket_name}"
        )

        print(
            f"File Name: {file_key}"
        )

        # -----
        # Read File From S3
        # -----

        response = s3.get_object(
            Bucket=bucket_name,
            Key=file_key
        )

        file_content = (
```

```

        response["Body"]
        .read()
        .decode("utf-8")
    )

    # -----
    # Invalid JSON Handling
    # -----

    try:

        students = json.loads(
            file_content,
            parse_float=Decimal
        )

    except json.JSONDecodeError:

        print(
            "Invalid JSON File"
        )

        return {

            "statusCode": 400,

            "message":
                "Invalid JSON File"
        }

    # -----
    # Required Fields
    # -----

    required_fields = [

        "student_id",

        "weekly_self_study_hours",

```

```

        "attendance_percentage",

        "class_participation",

        "total_score",

        "grade"
    ]

    # -----
    # Batch Processing
    # -----

    if len(students) > 500:

        print(
            "Using Batch Processing"
        )

        with table.batch_writer() as batch:

            for student in students:

                # -----
                # Missing Fields
                # -----

                missing_fields = [

                    field

                    for field
                    in required_fields

                    if field
                    not in student
                ]

                if missing_fields:

```

```

        print(

            f"Student skipped. "
            f"Missing fields: "
            f"{missing_fields}"

        )

        continue

# -----
# Duplicate Check
# -----

existing_student = (
    table.get_item(

        Key={
            "student_id":
                str(
                    student[
                        "student_id"
                    ]
                )
        }
    )
)

if (
    "Item"
    in existing_student
):

    print(

        f"Duplicate Student: "
        f"{student['student_id']}"

    )

    continue

```

```
# -----  
# Business Logic  
# -----  
  
student[  
    "performance_category"  
] = (  
    calculate_performance_category(  
        float(  
            student[  
                "total_score"  
            ]  
        )  
    )  
)  
  
student[  
    "at_risk"  
] = (  
    calculate_risk(  
        float(  
            student[  
                "attendance_percentage"  
            ]  
        ),  
        float(  
            student[  
                "total_score"  
            ]  
        )  
    )  
)  
  
student[  
    "student_id"  
] = str(  
    student[  
        "student_id"  
    ]  
)
```

```

        batch.put_item(
            Item=student
        )

        print(
            f"Inserted "
            f"{student['student_id']}"
        )

    else:

        print(
            "Using Normal Insert"
        )

    for student in students:

        # -----
        # Missing Fields
        # -----

        missing_fields = [

            field

            for field
            in required_fields

            if field
            not in student
        ]

        if missing_fields:

            print(

                f"Student skipped. "
                f"Missing fields: "
                f"{missing_fields}"
            )

```

```

        )

        continue

# -----
# Duplicate Check
# -----

existing_student = (
    table.get_item(

        Key={
            "student_id":
            str(
                student[
                    "student_id"
                ]
            )
        }
    )
)

if (
    "Item"
    in existing_student
):

    print(

        f"Duplicate Student: "
        f"{student['student_id']}"
    )

    continue

# -----
# Business Logic
# -----

student[

```

```

        "performance_category"
    ] = (
        calculate_performance_category(
            float(
                student[
                    "total_score"
                ]
            )
        )
    )

student[
    "at_risk"
] = (
    calculate_risk(
        float(
            student[
                "attendance_percentage"
            ]
        ),
        float(
            student[
                "total_score"
            ]
        )
    )
)

student[
    "student_id"
] = str(
    student[
        "student_id"
    ]
)

table.put_item(
    Item=student
)

```

```

        print(
            f"Inserted "
            f"{student['student_id']}"
        )

    return {

        "statusCode": 200,

        "message":
            "File processed successfully"
    }

except Exception as e:

    import traceback

    print(
        "ERROR OCCURRED"
    )

    print(
        traceback.format_exc()
    )

    raise e

```

What this code now satisfies

Requirement	Status
Performance Category	✓
Student Risk Detection	✓
Invalid JSON Handling	✓

Missing Fields Handling ✓

Duplicate Student Handling ✓

Batch Processing (>500) ✓

DynamoDB Insert ✓

S3 Trigger Processing ✓

Part 1 to Part 8 Workflow (Data Ingestion Pipeline)

Student Dataset (JSON)



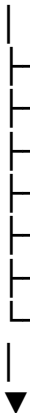
S3 Bucket (input/)



S3 ObjectCreated Event



AWS Lambda Triggered



Read JSON file from S3

Parse JSON records

Validate data

Calculate performance_category

Calculate at_risk

Check duplicates

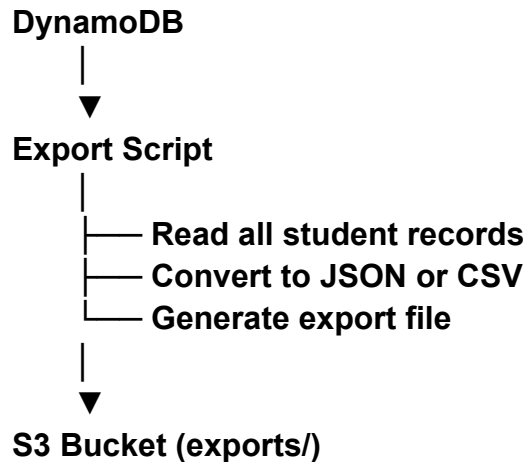
Insert records into DynamoDB



student_performance Table

Part 9 Workflow (Data Export Pipeline)

Part 9 is the reverse process.



The system uses an event-driven serverless architecture. When a JSON file is uploaded to the S3 input folder, S3 generates an **ObjectCreated** event that triggers a Lambda function. The Lambda function reads and validates the file, calculates **performance_category** and **at_risk** fields, and stores the student records in DynamoDB. For reporting purposes, an export module reads data from DynamoDB, generates JSON or CSV files, and uploads them to the exports folder in S3.